

Step-by-Step REST API Development Tutorial

```
backend-3/
├── config/
│   └── db.mongo.js          # MongoDB connection & readiness check
├── controllers/
│   ├── authController.js    # Register, login, profile & password management
│   ├── healthController.js  # Health check & version endpoints
│   ├── productController.js # Product CRUD, filters, search & pagination
│   └── userController.js    # Admin user management & RBAC operations
├── middlewares/
│   ├── authMiddleware.js    # JWT verification & role authorization
│   ├── errorHandler.js     # Centralized error handler & status mapper
│   ├── rateLimiter.js       # In-memory IP rate limiter
│   └── validateRequest.js   # Request body & payload validator
├── models/
│   ├── productModel.js      # Product Mongoose schema & relationship
│   └── userModel.js         # User Mongoose schema & bcrypt password hashing
├── routes/
│   ├── authRoutes.js        # /api/auth routes
│   ├── healthRoutes.js      # /api/health and /api/version routes
│   ├── productRoutes.js     # /api/products routes
│   └── userRoutes.js         # /api/users routes
├── utils/
│   ├── appVersion.js        # App metadata & versioning
│   └── jwt.js               # JWT token generator
├── .env                     # Environment variables
├── package.json             # Project dependencies & npm scripts
├── seeder.js                # Database seeder (import & wipe demo data)
├── server.js                # Main Express server entry point
├── TEST_CASES.md            # Test matrix & documentation
└── test_runner.js           # Automated 21-case security & RBAC test runner
```

Step 1: Initialize Project & Install Packages

1.1 Initialize Node.js Application

Open terminal in your project directory:

```
npm init -y
```

1.2 Install Required Dependencies

Install the runtime dependencies and development tools:

```
npm install express mongoose dotenv cors helmet morgan jsonwebtoken bcryptjs
npm install -D nodemon
```

Package	Purpose
<code>express</code>	Web framework for routing and middleware handling
<code>mongoose</code>	ODM for MongoDB schema modelling and validation
<code>dotenv</code>	Loads environment variables from <code>.env</code>
<code>jsonwebtoken</code>	Issues and verifies signed JWT tokens
<code>bcryptjs</code>	Hashes and salts passwords securely
<code>helmet</code>	Sets HTTP security response headers
<code>cors</code>	Configures Cross-Origin Resource Sharing
<code>morgan</code>	HTTP request logging for development & production
<code>nodemon</code>	Automatically restarts the server during development

1.3 Configure [package.json]

Configure the npm scripts:

```
{
  "name": "backend-3",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "seed:import": "node seeder.js -i",
  }
}
```

```
    "seed:destroy": "node seeder.js -d",  
    "test": "node test_runner.js"  
  }  
}
```

Step 2: Configure Environment Variables ([.env])

Create a `.env` file in the root directory:

```
PORT=5000  
NODE_ENV=development  
MONGODB_URI=  
JWT_SECRET=super_secret_jwt_key_mern_student_2026_secure  
JWT_EXPIRE=30d
```

Step 3: MongoDB Configuration

Create database connection logic with graceful fallback handling if the database is unavailable.

```
// config/db.mongo.js  
const mongoose = require('mongoose');  
require('dotenv').config();  
  
const isMongoConfigured = () => {  
  return (  
    typeof process.env.MONGODB_URI === 'string' &&  
    process.env.MONGODB_URI.trim() !== '' &&  
    !process.env.MONGODB_URI.includes('<')  
  );  
};  
  
const connectDB = async () => {  
  if (!isMongoConfigured()) {  
    console.warn('Warning: MONGODB_URI is not configured. Sta
```

```

    return without DB.');
```

```

    return;
  }
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log('Success: MongoDB Atlas Connected Successfully');
  } catch (error) {
    console.error('Error: MongoDB Connection Error:', error.message);
  }
};

const ensureDatabase = (res) => {
  if (mongoose.connection.readyState !== 1) {
    res.status(503).json({
      success: false,
      error: 'Database is not configured or unavailable. Add a valid MONGODB_URI.',
    });
    return false;
  }
  return true;
};

module.exports = { connectDB, ensureDatabase, isMongoConfigured };

```

Step 4: Utilities

4.1 JWT Utility ([utils/jwt.js])

Generates signed JSON Web Tokens containing user ID and role:

```
// utils/jwt.js
const jwt = require('jsonwebtoken');

const generateToken = (id, role) => {
  return jwt.sign(
    { id, role },
    process.env.JWT_SECRET || 'fallback_secret_key_change_in_production',
    { expiresIn: process.env.JWT_EXPIRE || '30d' }
  );
};

module.exports = { generateToken };
```

4.2 App Version

```
// utils/appVersion.js
const APP_VERSION = '2.0.0';
const DEPLOYED_AT = new Date().toISOString();

module.exports = { APP_VERSION, DEPLOYED_AT };
```

Step 5: Data Models

5.1 User Model

Handles user schema, RBAC roles (`user`, `moderator`, `admin`), pre-save bcrypt password hashing, password matching, and hides passwords when returning JSON.

```
// models/userModel.js
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
```

```

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Please provide a name'],
      trim: true,
      maxlength: [50, 'Name cannot exceed 50 characters'],
    },
    email: {
      type: String,
      required: [true, 'Please provide an email address'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, 'Please provide a
valid email address'],
    },
    password: {
      type: String,
      required: [true, 'Please provide a password'],
      minlength: [6, 'Password must be at least 6 characters
long'],
      select: false,
    },
    role: {
      type: String,
      enum: {
        values: ['user', 'moderator', 'admin'],
        message: '{VALUE} is not a valid role',
      },
      default: 'user',
    },
    isActive: {
      type: Boolean,
      default: true,
    },
  },

```

```

    },
    {
      timestamps: true,
      toJSON: {
        transform(doc, ret) {
          delete ret.password;
          delete ret.__v;
          return ret;
        },
      },
    },
  ],
};

// Pre-save hook to hash password before persisting
userSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// Compare entered password with hashed password
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);

```

5.2 Product Model

```

// models/productModel.js
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema(

```

```

{
  name: {
    type: String,
    required: [true, 'Please provide a product name'],
    trim: true,
    maxLength: [100, 'Product name cannot exceed 100 characters'],
  },
  description: {
    type: String,
    default: '',
    trim: true,
    maxLength: [1000, 'Description cannot exceed 1000 characters'],
  },
  price: {
    type: Number,
    required: [true, 'Please provide a price'],
    min: [0, 'Price cannot be negative'],
  },
  category: {
    type: String,
    enum: {
      values: ['electronics', 'clothing', 'food', 'books', 'other'],
      message: '{VALUE} is not a valid product category',
    },
    default: 'other',
  },
  inStock: {
    type: Boolean,
    default: true,
  },
  quantity: {
    type: Number,
    default: 0,
  },
}

```

```

        min: [0, 'Quantity cannot be negative'],
      },
      createdBy: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
      },
    },
  },
  {
    timestamps: true,
    toJSON: {
      transform(doc, ret) {
        delete ret.__v;
        return ret;
      },
    },
  },
}
);

module.exports = mongoose.model('Product', productSchema);

```

Step 6: Middlewares

6.1 Authentication & Authorization

Verifies JWT Bearer token and enforces role-based access permissions.

```

// middlewares/authMiddleware.js
const jwt = require('jsonwebtoken');
const User = require('../models/userModel');
const { ensureDatabase } = require('../config/db.mongo');

const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.
    startsWith('Bearer ')) {

```

```

    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return res.status(401).json({
      success: false,
      error: 'Not authorized to access this route. Token missing.',
    });
  }

  if (!ensureDatabase(res)) return;

  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET || 'fallback_secret_key_change_in_production'
    );

    const user = await User.findById(decoded.id);
    if (!user) {
      return res.status(401).json({
        success: false,
        error: 'User belonging to this token no longer exists.',
      });
    }

    if (!user.isActive) {
      return res.status(403).json({
        success: false,
        error: 'Your account has been deactivated. Please contact administrator.',
      });
    }
  }

```

```

    req.user = user;
    next();
  } catch (error) {
    if (error.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        error: 'Authentication token has expired. Please log
in again.',
      });
    }
    return res.status(401).json({
      success: false,
      error: 'Invalid authentication token.',
    });
  }
};

const authorize = (...roles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ success: false, error: 'Not
authorized.' });
    }
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        error: `User role '${req.user.role}' is unauthorized
to access this resource.`,
      });
    }
    next();
  };
};

```

```
module.exports = { protect, authorize };
```

6.2 Request Validation

```
// middlewares/validateRequest.js
const validateRequest = (requiredFields = []) => {
  return (req, res, next) => {
    if (!req.body || typeof req.body !== 'object') {
      return res.status(400).json({ success: false, error: 'Invalid request body' });
    }

    const missingFields = requiredFields.filter((field) => {
      const val = req.body[field];
      return val === undefined || val === null || String(val).trim() === '';
    });

    if (missingFields.length > 0) {
      return res.status(400).json({
        success: false,
        error: `Missing required fields: ${missingFields.join(', ')}`,
        missingFields,
      });
    }

    if (requiredFields.includes('email') && req.body.email) {
      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      if (!emailRegex.test(String(req.body.email).trim())) {
        return res.status(400).json({
          success: false,
          error: 'Please provide a valid email address format',
        });
      }
    }
  };
}
```

```

    });
  }
}

if (requiredFields.includes('password') && req.body.password) {
  if (String(req.body.password).length < 6) {
    return res.status(400).json({
      success: false,
      error: 'Password must be at least 6 characters long',
    });
  }
}

next();
};
};

module.exports = validateRequest;

```

6.3 Global Error Handler

Formats CastErrors, Duplicate Key (11000) errors, validation errors, and JWT errors cleanly:

```

// middlewares/errorHandler.js
const errorHandler = (err, req, res, next) => {
  let error = { ...err };
  error.message = err.message;

  if (process.env.NODE_ENV !== 'test') {
    console.error(`[API Error] ${err.name}: ${err.message}`);
  }

  // Mongoose Bad ObjectId (CastError)

```

```

    if (err.name === 'CastError') {
      return res.status(400).json({
        success: false,
        error: `Resource not found. Invalid ID format: '${err.v
alue}'`,
      });
    }

    // Mongoose Duplicate Key (11000)
    if (err.code === 11000) {
      const field = err.keyPattern ? Object.keys(err.keyPatter
n)[0] : 'field';
      return res.status(409).json({
        success: false,
        error: `Duplicate value entered for '${field}'. This va
lue already exists.`,
      });
    }

    // Mongoose Validation Error
    if (err.name === 'ValidationError') {
      const messages = Object.values(err.errors).map((val) => v
al.message);
      return res.status(400).json({
        success: false,
        error: 'Validation Error',
        details: messages,
      });
    }

    // Default Fallback
    const statusCode = err.statusCode || (res.statusCode === 20
0 ? 500 : res.statusCode || 500);
    res.status(statusCode).json({
      success: false,
      error: error.message || 'Internal Server Error',
    });
  }
}

```

```

    ...(process.env.NODE_ENV === 'development' && { stack: err.stack } ),
  });
};

module.exports = errorHandler;

```

6.4 Rate Limiter

Protects against brute force and DDoS attacks:

```

// middlewares/rateLimiter.js
const rateLimitStore = new Map();

const rateLimiter = (maxRequests = 100, windowMs = 15 * 60 * 1000) => {
  return (req, res, next) => {
    const clientIP = req.ip || req.connection.remoteAddress;
    const now = Date.now();

    if (!rateLimitStore.has(clientIP)) {
      rateLimitStore.set(clientIP, { count: 1, startTime: now });
    }

    return next();
  }

  const clientData = rateLimitStore.get(clientIP);

  if (now - clientData.startTime > windowMs) {
    rateLimitStore.set(clientIP, { count: 1, startTime: now });
  }

  return next();
}

clientData.count += 1;

```

```

    if (clientData.count > maxRequests) {
      return res.status(429).json({
        success: false,
        error: 'Too many requests. Please try again later.',
        retryAfterMs: windowMs - (now - clientData.startTime),
      });
    }

    next();
  };
};

module.exports = rateLimiter;

```

Step 7: Controllers

7.1 Auth Controller

```

// controllers/authController.js
const User = require('../models/userModel');
const { generateToken } = require('../utils/jwt');
const { ensureDatabase } = require('../config/db.mongo');

// POST /api/auth/register
const register = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const { name, email, password, role } = req.body;
    const existingUser = await User.findOne({ email: email.toLowerCase().trim() : '' });
    if (existingUser) {
      return res.status(409).json({
        success: false,

```

```

        error: 'An account with this email address already exists',
    });
}

const userRole = role && ['user', 'moderator', 'admin'].includes(role) ? role : 'user';
const user = await User.create({ name, email, password, role: userRole });
const token = generateToken(user._id, user.role);

res.status(201).json({
    success: true,
    message: 'User registered successfully',
    token,
    data: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        isActive: user.isActive,
        createdAt: user.createdAt,
    },
});
} catch (error) {
    next(error);
}
};

// POST /api/auth/login
const login = async (req, res, next) => {
    if (!ensureDatabase(res)) return;
    try {
        const { email, password } = req.body;
        if (!email || !password) {
            return res.status(400).json({ success: false, error: 'P

```

```

    'Please provide email and password' });
  }

  const user = await User.findOne({ email: email.toLowerCase().trim() }).select('+password');
  if (!user || !(await user.matchPassword(password))) {
    return res.status(401).json({ success: false, error: 'Invalid email or password credentials' });
  }

  if (!user.isActive) {
    return res.status(403).json({ success: false, error: 'Account is deactivated. Please contact support.' });
  }

  const token = generateToken(user._id, user.role);
  res.json({
    success: true,
    message: 'Login successful',
    token,
    data: {
      id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      isActive: user.isActive,
      createdAt: user.createdAt,
    },
  });
} catch (error) {
  next(error);
}
};

// GET /api/auth/me
const getMe = async (req, res, next) => {

```

```

    res.json({ success: true, data: req.user });
  };

// PUT /api/auth/updatedetails
const updateDetails = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const fieldsToUpdate = {};
    if (req.body.name) fieldsToUpdate.name = req.body.name;
    if (req.body.email) fieldsToUpdate.email = req.body.email;

    if (fieldsToUpdate.email && fieldsToUpdate.email.toLowerCase() !== req.user.email) {
      const emailExists = await User.findOne({ email: fieldsToUpdate.email.toLowerCase() });
      if (emailExists) {
        return res.status(409).json({ success: false, error: 'Email is already in use by another account' });
      }
    }

    const user = await User.findByIdAndUpdate(req.user.id, fieldsToUpdate, {
      returnDocument: 'after',
      runValidators: true,
    });

    res.json({ success: true, message: 'Profile updated successfully', data: user });
  } catch (error) {
    next(error);
  }
};

// PUT /api/auth/updatepassword

```

```

const updatePassword = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const { currentPassword, newPassword } = req.body;
    if (!currentPassword || !newPassword) {
      return res.status(400).json({ success: false, error: 'Please provide both current and new password' });
    }

    const user = await User.findById(req.user.id).select('+password');
    if (!(await user.matchPassword(currentPassword))) {
      return res.status(401).json({ success: false, error: 'Current password is incorrect' });
    }

    user.password = newPassword;
    await user.save();
    const token = generateToken(user._id, user.role);

    res.json({ success: true, message: 'Password updated successfully', token });
  } catch (error) {
    next(error);
  }
};

module.exports = { register, login, getMe, updateDetails, updatePassword };

```

7.2 Product Controller

Supports search, filtering by price/category, sorting, pagination, and populate.

```

// controllers/productController.js
const Product = require('../models/productModel');

```

```

const { ensureDatabase } = require('../config/db.mongo');

// GET /api/products
const getAllProducts = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const {
      page = 1,
      limit = 10,
      category,
      inStock,
      minPrice,
      maxPrice,
      search,
      sortBy = 'createdAt',
      sortOrder = 'desc',
    } = req.query;

    const filter = {};
    if (category) filter.category = category;
    if (inStock !== undefined) filter.inStock = inStock ===
'true';

    if (minPrice || maxPrice) {
      filter.price = {};
      if (minPrice) filter.price.$gte = parseFloat(minPrice);
      if (maxPrice) filter.price.$lte = parseFloat(maxPrice);
    }

    if (search) {
      filter.$or = [
        { name: { $regex: search, $options: 'i' } },
        { description: { $regex: search, $options: 'i' } },
      ];
    }
  }

```

```

    const pageNum = Math.max(1, parseInt(page, 10) || 1);
    const limitNum = Math.max(1, Math.min(100, parseInt(limit, 10) || 10));
    const sort = { [sortBy]: sortOrder === 'asc' ? 1 : -1 };

    const products = await Product.find(filter)
      .populate('createdBy', 'name email role')
      .sort(sort)
      .skip((pageNum - 1) * limitNum)
      .limit(limitNum);

    const total = await Product.countDocuments(filter);

    res.json({
      success: true,
      data: products,
      pagination: {
        currentPage: pageNum,
        totalPages: Math.ceil(total / limitNum),
        totalRecords: total,
        limit: limitNum,
      },
    });
  } catch (error) {
    next(error);
  }
};

// POST /api/products
const createProduct = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const { name, description, price, category, inStock, quantity } = req.body;
    const product = new Product({
      name,

```

```

        description,
        price,
        category,
        inStock: inStock !== undefined ? inStock : true,
        quantity: quantity !== undefined ? quantity : 0,
        createdBy: req.user ? req.user._id : undefined,
    });

    await product.save();
    res.status(201).json({ success: true, message: 'Product created successfully', data: product });
  } catch (error) {
    next(error);
  }
};

// GET /api/products/:id
const getProductById = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const product = await Product.findById(req.params.id).populate('createdBy', 'name email role');
    if (!product) return res.status(404).json({ success: false, error: 'Product not found' });
    res.json({ success: true, data: product });
  } catch (error) {
    next(error);
  }
};

// PUT /api/products/:id
const updateProduct = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const product = await Product.findByIdAndUpdate(
      req.params.id,

```

```

    req.body,
    { returnDocument: 'after', runValidators: true }
  ).populate('createdBy', 'name email role');

  if (!product) return res.status(404).json({ success: false, error: 'Product not found' });
  res.json({ success: true, message: 'Product updated successfully', data: product });
} catch (error) {
  next(error);
}
};

// DELETE /api/products/:id
const deleteProduct = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) return res.status(404).json({ success: false, error: 'Product not found' });
    res.json({ success: true, message: 'Product deleted successfully' });
  } catch (error) {
    next(error);
  }
};

module.exports = { getAllProducts, createProduct, getProductById, updateProduct, deleteProduct };

```

7.3 User Management Controller

```

// controllers/userController.js
const User = require('../models/userModel');

```

```

const { ensureDatabase } = require('../config/db.mongo');

const getAllUsers = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const { page = 1, limit = 10, role, isActive, search } =
req.query;
    const filter = {};
    if (role) filter.role = role;
    if (isActive !== undefined) filter.isActive = isActive ==
= 'true';
    if (search) {
      filter.$or = [
        { name: { $regex: search, $options: 'i' } },
        { email: { $regex: search, $options: 'i' } },
      ];
    }

    const pageNum = Math.max(1, parseInt(page, 10) || 1);
    const limitNum = Math.max(1, Math.min(100, parseInt(limi
t, 10) || 10));

    const users = await User.find(filter)
      .sort({ createdAt: -1 })
      .skip((pageNum - 1) * limitNum)
      .limit(limitNum);

    const total = await User.countDocuments(filter);

    res.json({
      success: true,
      data: users,
      pagination: {
        currentPage: pageNum,
        totalPages: Math.ceil(total / limitNum),
        totalRecords: total,

```

```

        limit: limitNum,
      },
    ));
  } catch (error) {
    next(error);
  }
};

const createUser = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const { name, email, password, role, isActive } = req.body;

    const existingUser = await User.findOne({ email: email ?
    email.toLowerCase().trim() : '' });
    if (existingUser) {
      return res.status(409).json({ success: false, error: 'A
user with this email address already exists' });
    }

    const user = new User({
      name,
      email,
      password: password || 'DefaultPass123!',
      role: role || 'user',
      isActive: isActive !== undefined ? isActive : true,
    });

    await user.save();
    res.status(201).json({ success: true, message: 'User crea
ted successfully', data: user });
  } catch (error) {
    next(error);
  }
};

```

```

const getUserById = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ success: false,
error: 'User not found' });
    res.json({ success: true, data: user });
  } catch (error) {
    next(error);
  }
};

const updateUser = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const { name, email, role, isActive, password } = req.body;

    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ success: false,
error: 'User not found' });

    if (name) user.name = name;
    if (email) {
      const emailLower = email.toLowerCase().trim();
      if (emailLower !== user.email) {
        const emailExists = await User.findOne({ email: email
Lower });
        if (emailExists) return res.status(409).json({ succes
s: false, error: 'Email is already taken' });
        user.email = emailLower;
      }
    }
    if (role) user.role = role;
    if (isActive !== undefined) user.isActive = isActive;
    if (password) user.password = password;
  }
};

```

```

    await user.save();
    res.json({ success: true, message: 'User updated successfully', data: user });
  } catch (error) {
    next(error);
  }
};

const deleteUser = async (req, res, next) => {
  if (!ensureDatabase(res)) return;
  try {
    const user = await User.findByIdAndDelete(req.params.id);
    if (!user) return res.status(404).json({ success: false, error: 'User not found' });
    res.json({ success: true, message: 'User deleted successfully' });
  } catch (error) {
    next(error);
  }
};

module.exports = { getAllUsers, createUser, getUserById, updateUser, deleteUser };

```

7.4 Health Controller

```

// controllers/healthController.js
const mongoose = require('mongoose');
const { APP_VERSION, DEPLOYED_AT } = require('../utils/appVersion');

const getHealth = (req, res) => {
  const dbState = mongoose.connection.readyState;
  const dbStatusMap = { 0: 'disconnected', 1: 'connected', 2: 'connecting', 3: 'disconnecting' };

```

```

res.json({
  status: 'OK',
  message: 'Backend API is running',
  timestamp: new Date().toISOString(),
  environment: process.env.NODE_ENV || 'development',
  database: dbStatusMap[dbState] || 'unknown',
  uptime: process.uptime(),
  memoryUsage: {
    rss: `${(process.memoryUsage().rss / 1024 / 1024).toFixed(2)} MB`,
    heapUsed: `${(process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2)} MB`,
    heapTotal: `${(process.memoryUsage().heapTotal / 1024 / 1024).toFixed(2)} MB`,
  },
});

const getVersion = (req, res) => {
  res.json({
    success: true,
    version: APP_VERSION,
    deployedAt: DEPLOYED_AT,
    nodeVersion: process.version,
    platform: process.platform,
  });
};

module.exports = { getHealth, getVersion };

```

Step 8: Routes

8.1 Auth Routes

```
// routes/authRoutes.js
const express = require('express');
const router = express.Router();
const authController = require('../controllers/authController');
const { protect } = require('../middlewares/authMiddleware');
const validateRequest = require('../middlewares/validateRequest');

router.post('/register', validateRequest(['name', 'email', 'password']), authController.register);
router.post('/login', validateRequest(['email', 'password']), authController.login);
router.get('/me', protect, authController.getMe);
router.put('/updatedetails', protect, authController.updateDetails);
router.put('/updatepassword', protect, validateRequest(['currentPassword', 'newPassword']), authController.updatePassword);

module.exports = router;
```

8.2 Product Routes with RBAC

```
// routes/productRoutes.js
const express = require('express');
const router = express.Router();
const productController = require('../controllers/productController');
const { protect, authorize } = require('../middlewares/authMiddleware');
const validateRequest = require('../middlewares/validateRequest');
```

```

// Public
router.get('/', productController.getAllProducts);
router.get('/:id', productController.getProductById);

// Admin & Moderator
router.post('/', protect, authorize('admin', 'moderator'), validateRequest(['name', 'price']), productController.createProduct);
router.put('/:id', protect, authorize('admin', 'moderator'), productController.updateProduct);

// Admin Only
router.delete('/:id', protect, authorize('admin'), productController.deleteProduct);

module.exports = router;

```

8.3 User Management Routes

```

// routes/userRoutes.js
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');
const { protect, authorize } = require('../middlewares/authMiddleware');
const validateRequest = require('../middlewares/validateRequest');

// All endpoints require Admin access
router.use(protect);
router.use(authorize('admin'));

router.get('/', userController.getAllUsers);
router.post('/', validateRequest(['name', 'email', 'password']), userController.createUser);

```

```
d']], userController.createUser);  
router.get('/:id', userController.getUserById);  
router.put('/:id', userController.updateUser);  
router.delete('/:id', userController.deleteUser);  
  
module.exports = router;
```

8.4 Health Routes

```
// routes/healthRoutes.js  
const express = require('express');  
const router = express.Router();  
const healthController = require('../controllers/healthController');  
  
router.get('/health', healthController.getHealth);  
router.get('/version', healthController.getVersion);  
  
module.exports = router;
```

Step 9: Express Server Entry Point

Connects all middlewares, mounts routes, provides root documentation, and starts the server:

```
// server.js  
const express = require('express');  
const cors = require('cors');  
const helmet = require('helmet');  
const morgan = require('morgan');  
require('dotenv').config();  
  
const { connectDB } = require('./config/db.mongo');  
const errorHandler = require('./middlewares/errorHandler');
```

```

const rateLimiter = require('./middlewares/rateLimiter');

const healthRoutes = require('./routes/healthRoutes');
const authRoutes = require('./routes/authRoutes');
const userRoutes = require('./routes/userRoutes');
const productRoutes = require('./routes/productRoutes');

const app = express();

// Security Headers & Logging
app.use(helmet());
if (process.env.NODE_ENV !== 'test') {
  app.use(morgan(process.env.NODE_ENV === 'production' ? 'combined' : 'dev'));
}

// CORS Configuration
app.use(cors());

// Parsers & Rate Limiter
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(rateLimiter(200, 15 * 60 * 1000));

// Root Discovery
app.get('/', (req, res) => {
  res.json({
    success: true,
    message: 'Welcome to the REST API with Authentication & RBAC Authorization',
    endpoints: {
      health: '/api/health',
      version: '/api/version',
      auth: {
        register: 'POST /api/auth/register',
        login: 'POST /api/auth/login',
      }
    }
  });
});

```

```

        me: 'GET /api/auth/me',
      },
      users: { getAll: 'GET /api/users (Admin only)' },
      products: { getAll: 'GET /api/products' },
    },
  });
});

// Mount Routes
app.use('/api', healthRoutes);
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/products', productRoutes);

// 404 Fallback
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: `Route ${req.method} ${req.originalUrl} not found`
  });
});

// Central Error Handler
app.use(errorHandler);

const PORT = process.env.PORT || 5000;
connectDB();

if (require.main === module) {
  app.listen(PORT, '0.0.0.0', () => {
    console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
  });
}

```

```
module.exports = app;
```

Step 10: Database Seeder

The seeder creates pre-configured accounts and catalog products for testing:

```
// seeder.js
const mongoose = require('mongoose');
require('dotenv').config();

const User = require('./models/userModel');
const Product = require('./models/productModel');

const users = [
  { name: 'System Administrator', email: 'admin@example.com',
    password: 'AdminPassword123!', role: 'admin', isActive: true
  },
  { name: 'Content Moderator', email: 'moderator@example.com',
    password: 'ModPassword123!', role: 'moderator', isActive: true
  },
  { name: 'Regular Customer', email: 'user@example.com', password: 'UserPassword123!',
    role: 'user', isActive: true },
  { name: 'Deactivated User', email: 'inactive@example.com',
    password: 'InactivePassword123!', role: 'user', isActive: false
  },
];

const sampleProducts = [
  { name: 'MacBook Pro 16" M3 Max', description: 'Apple M3 Max chip, 64GB RAM, 1TB SSD',
    price: 3499.99, category: 'electronics', inStock: true, quantity: 18 },
  { name: 'Sony WH-1000XM5 Wireless Headphones', description: 'Industry-leading noise canceling',
    price: 398.00, category: 'electronics', inStock: true, quantity: 45 },
];
```

```

    { name: 'Clean Code Book', description: 'Handbook of Agile
Software Craftsmanship', price: 44.95, category: 'books', inS
tock: true, quantity: 80 },
];

const importData = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    await User.deleteMany();
    await Product.deleteMany();

    const createdUsers = [];
    for (const u of users) {
      const userDoc = new User(u);
      await userDoc.save();
      createdUsers.push(userDoc);
    }

    const admin = createdUsers.find((u) => u.role === 'admin');
    const productsWithCreator = sampleProducts.map((p) => ({
...p, createdBy: admin._id }));
    await Product.insertMany(productsWithCreator);

    console.log('✅ Demo Data Imported Successfully!');
    process.exit(0);
  } catch (err) {
    console.error(err);
    process.exit(1);
  }
};

const destroyData = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    await User.deleteMany();

```

```

    await Product.deleteMany();
    console.log('✅ All data destroyed!');
    process.exit(0);
  } catch (err) {
    console.error(err);
    process.exit(1);
  }
};

if (process.argv[2] === '-d') {
  destroyData();
} else {
  importData();
}

```

Step 11: Automated Test Runner

The test runner validates **21 automated end-to-end security and RBAC scenarios**:

#	Test Scenario	Method & Route	Expected
01	API Discovery Directory	GET /	200 OK
02	System Health Check	GET /api/health	200 OK
03	Missing Required Fields	POST /api/auth/register	400 Bad Request
04	Register Standard User	POST /api/auth/register	201 Created
05	Duplicate Email Blocked	POST /api/auth/register	409 Conflict
06	Register Moderator User	POST /api/auth/register	201 Created
07	Register Admin User	POST /api/auth/register	201 Created
08	Wrong Password Rejection	POST /api/auth/login	401 Unauthorized

#	Test Scenario	Method & Route	Expected
09	Valid Login & Token Issue	POST /api/auth/login	200 OK + JWT
10	Get User Profile (/me)	GET /api/auth/me	200 OK
11	Access without Bearer Token	GET /api/auth/me	401 Unauthorized
12	Update Profile Details	PUT /api/auth/updatedetails	200 OK
13	RBAC: User blocked from Admin routes	GET /api/users	403 Forbidden
14	RBAC: Admin allowed access to users	GET /api/users	200 OK
15	Public Product Catalog Browsing	GET /api/products	200 OK
16	RBAC: Standard user cannot create product	POST /api/products	403 Forbidden
17	RBAC: Moderator allowed to create product	POST /api/products	201 Created
18	Fetch Product by ID (Public)	GET /api/products/:id	200 OK
19	RBAC: Moderator cannot delete product	DELETE /api/products/:id	403 Forbidden
20	RBAC: Admin allowed to delete product	DELETE /api/products/:id	200 OK
21	Undefined Route Fallback	GET /api/undefined-route	404 Not Found

Step 12: How to Run and Test the Project

```
# 1. Start development server with live reload
npm run dev
```

```
# 2. Seed demo accounts and catalog data
```

```
npm run seed:import
```

```
# 3. Run all 21 automated integration and RBAC test cases
```

```
npm test
```

```
# 4. Clean/destroy database collections when needed
```

```
npm run seed:destroy
```